aws
English ▼ Preferences ▼ Contact Us Feedback
Get started Service guides Developer tools AI resources
Search in this guide
Create an AWS Account

Elastic Load Balancing
Application Load Balancers

- What is an Application Load Balancer?
- ▶ Application Load Balancers
- ▶ Listeners and rules
- ▶ Target groups
- ▼ Monitor your load balancers
  - CloudWatch metrics
  - ▶ Access logs
  - ▶ Connection logs
  - ▶ Health check logs
  - Request tracing
- Troubleshoot your load balancers
- Quotas
- Document history

Documentation > Elastic Load Balancing > Application Load Balancers

CloudWatch metrics for your Application Load Balancer

PDF RSS Markdown Focus mode

Elastic Load Balancing publishes data points to Amazon CloudWatch for your load balancers and your targets. CloudWatch enables you to retrieve statistics about those data points as an ordered set of time-series data, known as *metrics*. Think of a metric as a variable to monitor, and the data points as the values of that variable over time. For example, you can monitor the total number of healthy targets for a load balancer over a specified time period. Each data point has an associated time stamp and an optional unit of measurement.

You can use metrics to verify that your system is performing as expected. For example, you can create a CloudWatch alarm to monitor a specified metric and initiate an action (such as sending a notification to an email address) if the metric goes outside what you consider an acceptable range.

Elastic Load Balancing reports metrics to CloudWatch only when requests are flowing through the load balancer. If there are requests flowing through the load balancer, Elastic Load Balancing measures and sends its metrics in 60-second intervals. If there are no requests flowing through the load balancer or no data for a metric, the metric is not reported.

Metrics for Application Load Balancers exclude health check requests.

For more information, see the Amazon CloudWatch User Guide.

Contents

- Application Load Balancer metrics
- Metric dimensions for Application Load Balancers
- Statistics for Application Load Balancer metrics
- View CloudWatch metrics for your load balancer

On this page
Application Load Balancer metrics
Metric dimensions for Application Load Balancers
Statistics for Application Load Balancer metrics
View CloudWatch metrics for your load balancer

▼ Recommended tasks

How to
Configure target tracking scaling policies
Configure Application Load Balancer access logs in S3 bucket
Troubleshoot issues with Application Load Balancers
Configure alarms to detect high rates of API Gateway errors
Create a table for Application Load Balancer access logs

Learn about
Understand types of metrics for monitoring Lambda functions
Understand CloudWatch metrics for health checks
Understand how to use Application Load Balancer access logs
Understand CloudWatch metrics for Network Load Balancer
Understand CloudWatch metrics available for EC2 instances

Did this page help you?
Yes No
Provide feedback

Application Load Balancer metrics

- Load balancers
- LCUs
- Targets
- Target group health
- Lambda functions
- User authentication
- Target Optimizer

The `AWS/ApplicationELB` namespace includes the following metrics for load balancers.

Metric	Description
`ActiveConnectionCount`	The total number of concurrent TCP connections active from clients to the load balancer and from the load balancer to targets. **Reporting criteria**: There is a nonzero value **Statistics**: The only meaningful statistic is `Sum`. **Dimensions** • `LoadBalancer` • `AvailabilityZone`, `LoadBalancer`
`AppCookieNonStickinessCount`	The number of requests where the load balancer chose a new target because it couldn't use an existing application-based sticky session. For example, the request was the first request from a new client and no application stickiness cookie was presented, a stickiness cookie was presented but it could not be decrypted, referred to a target that was no longer registered with the target group, or the stickiness cookie was malformed or expired. **Reporting criteria**: Application-based stickiness is enabled on the target group. **Statistics**: The only meaningful statistic is `Sum`. **Dimensions** • `LoadBalancer` • `AvailabilityZone`, `LoadBalancer`
`BYoIPUtilPercentage`	The percentage of usage from the IP pool. **Reporting criteria**: BYoIP is enabled on the load balancer. **Statistics**: The only meaningful statistic is `Average`. **Dimensions** • `LoadBalancer`, `TargetGroup` • `LoadBalancer`, `TargetGroup`, `AvailabilityZone`
`ClientTLSNegotiationErrorCount`	The number of TLS connections initiated by the client that did not establish a session with the load balancer due to a TLS error. Possible causes include a mismatch of ciphers or protocols or the client failing to verify the server certificate and closing the connection. **Reporting criteria**: There is a nonzero value **Statistics**: The only meaningful statistic is `Sum`. **Dimensions** • `LoadBalancer` • `AvailabilityZone`, `LoadBalancer`
`DesyncMitigationMode_NonCompliant_Request_Count`	The number of requests that do not comply with RFC 7230. **Reporting criteria**: There is a nonzero value **Statistics**: The most useful statistic is `Sum`. **Dimensions** • `LoadBalancer` • `AvailabilityZone`, `LoadBalancer`
`DroppedInvalidHeaderRequestCount`	The number of requests where the load balancer removed HTTP headers with header fields that are not valid before routing the request. The load balancer removes these headers only if the `routing.http.drop_invalid_header_fields.enabled` attribute is set to `true`. **Reporting criteria**: There is a nonzero value **Statistics**: All **Dimensions** • `AvailabilityZone`, `LoadBalancer`
`ExcessiveLowReputationPackets`	The number of packets from known malicious sources that exceeded the rate limit. This represents packets that would have been dropped if the load balancer were in active blocking mode. **Reporting criteria**: There is a nonzero value. **Statistics**: The only meaningful statistic is `Sum`. **Dimensions** • `LoadBalancer` • `AvailabilityZone`, `LoadBalancer`
`ForwardedInvalidHeaderRequestCount`	The number of requests routed by the load balancer that had HTTP headers with header fields that are not valid. The load balancer forwards requests with these headers only if the `routing.http.drop_invalid_header_fields.enabled` attribute is set to `false`. **Reporting criteria**: Always reported **Statistics**: All **Dimensions** • `AvailabilityZone`, `LoadBalancer`
`GrpcRequestCount`	The number of gRPC requests processed over IPv4 and IPv6. **Reporting criteria**: There is a nonzero value **Statistics**: The most useful statistic is `Sum`. `Minimum`, `Maximum`, and `Average` all return 1. **Dimensions** • `LoadBalancer`, `TargetGroup` • `AvailabilityZone`, `LoadBalancer`, `TargetGroup` • `TargetGroup` • `AvailabilityZone`, `TargetGroup`
`HTTP_Fixed_Response_Count`	The number of fixed-response actions that were successful. **Reporting criteria**: There is a nonzero value **Statistics**: The only meaningful statistic is `Sum`. **Dimensions** • `LoadBalancer` • `AvailabilityZone`, `LoadBalancer`
`HTTP_Redirect_Count`	The number of redirect actions that were successful. **Reporting criteria**: There is a nonzero value **Statistics**: The only meaningful statistic is `Sum`. **Dimensions** • `LoadBalancer` • `AvailabilityZone`, `LoadBalancer`
`HTTP_Redirect_Url_Limit_Exceeded_Count`	The number of redirect actions that couldn't be completed because the URL in the response location header is larger than 8K. **Reporting criteria**: There is a nonzero value **Statistics**: The only meaningful statistic is `Sum`. **Dimensions** • `LoadBalancer` • `AvailabilityZone`, `LoadBalancer`
`HTTPCode_ELB_3XX_Count`	The number of HTTP 3XX redirection codes that originate from the load balancer. This count does not include response codes generated by targets. **Reporting criteria**: There is a nonzero value **Statistics**: The only meaningful statistic is `Sum`. **Dimensions** • `LoadBalancer` • `AvailabilityZone`, `LoadBalancer`
`HTTPCode_ELB_4XX_Count`	The number of HTTP 4XX client error codes that originate from the load balancer. This count does not include response codes generated by targets. Client errors are generated when requests are malformed or incomplete. These requests were not received by the target, other than in the case where the load balancer returns an HTTP 460 error code. This count does not include any response codes generated by the targets. **Reporting criteria**: There is a nonzero value **Statistics**: The most useful statistic is `Sum`. `Minimum`, `Maximum`, and `Average` all return 1. **Dimensions** • `LoadBalancer` • `AvailabilityZone`, `LoadBalancer`
`HTTPCode_ELB_5XX_Count`	The number of HTTP 5XX server error codes that originate from the load balancer. This count does not include any response codes generated by the targets. **Reporting criteria**: There is a nonzero value **Statistics**: The most useful statistic is `Sum`. `Minimum`, `Maximum`, and `Average` all return 1. **Dimensions** • `LoadBalancer` • `AvailabilityZone`, `LoadBalancer`
`HTTPCode_ELB_500_Count`	The number of HTTP 500 error codes that originate from the load balancer. **Reporting criteria**: There is a nonzero value **Statistics**: The only meaningful statistic is `Sum`. **Dimensions** • `LoadBalancer` • `AvailabilityZone`, `LoadBalancer`
`HTTPCode_ELB_502_Count`	The number of HTTP 502 error codes that originate from the load balancer. **Reporting criteria**: There is a nonzero value **Statistics**: The only meaningful statistic is `Sum`. **Dimensions** • `LoadBalancer` • `AvailabilityZone`, `LoadBalancer`
`HTTPCode_ELB_503_Count`	The number of HTTP 503 error codes that originate from the load balancer. **Reporting criteria**: There is a nonzero value **Statistics**: The only meaningful statistic is `Sum`. **Dimensions** • `LoadBalancer` • `AvailabilityZone`, `LoadBalancer`
`HTTPCode_ELB_504_Count`	The number of HTTP 504 error codes that originate from the load balancer. **Reporting criteria**: There is a nonzero value **Statistics**: The only meaningful statistic is `Sum`. **Dimensions** • `LoadBalancer` • `AvailabilityZone`, `LoadBalancer`
`IPv6ProcessedBytes`	The total number of bytes processed by the load balancer over IPv6. This count is included in `ProcessedBytes`. **Reporting criteria**: There is a nonzero value **Statistics**: The most useful statistic is `Sum`. **Dimensions** • `LoadBalancer` • `AvailabilityZone`, `LoadBalancer`
`IPv6RequestCount`	The number of IPv6 requests received by the load balancer. **Reporting criteria**: There is a nonzero value **Statistics**: The most useful statistic is `Sum`. `Minimum`, `Maximum`, and `Average` all return 1. **Dimensions** • `LoadBalancer` • `AvailabilityZone`, `LoadBalancer`
`LowReputationPacketsDropped`	The number of packets dropped from known malicious sources. This metric is recorded when a request is blocked by resource-level DDoS protection. **Reporting criteria**: There is a nonzero value **Statistics**: The only meaningful statistic is `Sum`. **Dimensions** • `LoadBalancer` • `AvailabilityZone`, `LoadBalancer`
`LowReputationRequestsDenied`	The number of HTTP requests denied with an HTTP 403 response. This metric is recorded when a request is blocked by resource-level DDoS protection. **Reporting criteria**: There is a nonzero value **Statistics**: The only meaningful statistic is `Sum`. **Dimensions** • `LoadBalancer` • `AvailabilityZone`, `LoadBalancer`
`NewConnectionCount`	The total number of new TCP connections established from clients to the load balancer and from the load balancer to targets. **Reporting criteria**: There is a nonzero value **Statistics**: The most useful statistic is `Sum`. **Dimensions** • `LoadBalancer` • `AvailabilityZone`, `LoadBalancer`
`NonStickyRequestCount`	The number of requests where the load balancer chose a new target because it couldn't use an existing sticky session. For example, the request was the first request from a new client and no stickiness cookie was presented, a stickiness cookie was presented but it did not specify a target that was registered with this target group, the stickiness cookie was malformed or expired, or an internal error prevented the load balancer from reading the stickiness cookie. **Reporting criteria**: Stickiness is enabled on the target group. **Statistics**: The only meaningful statistic is `Sum`. **Dimensions** • `LoadBalancer` • `AvailabilityZone`, `LoadBalancer`
`ProcessedBytes`	The total number of bytes processed by the load balancer over IPv4 and IPv6 (HTTP header and HTTP payload). This count includes traffic to and from clients and Lambda functions, traffic over Websocket connections, and traffic from an Identity Provider (IdP) if user authentication is enabled. **Reporting criteria**: There is a nonzero value **Statistics**: The most useful statistic is `Sum`. **Dimensions** • `LoadBalancer` • `AvailabilityZone`, `LoadBalancer`
`RejectedConnectionCount`	The number of connections that were rejected because the load balancer had reached its maximum number of connections. **Reporting criteria**: There is a nonzero value **Statistics**: The most useful statistic is `Sum`. **Dimensions** • `LoadBalancer` • `AvailabilityZone`, `LoadBalancer`
`RequestCount`	The number of requests processed over IPv4 and IPv6. This metric is only incremented for requests where the load balancer node was able to choose a target. Requests that are rejected before a target is chosen are not reflected in this metric. **Reporting criteria**: Reported if there are registered targets. **Statistics**: The most useful statistic is `Sum`. **Dimensions** • `LoadBalancer` • `LoadBalancer`, `AvailabilityZone` • `LoadBalancer`, `TargetGroup` • `LoadBalancer`, `AvailabilityZone`, `TargetGroup`
`RuleEvaluations`	The number of rules evaluated by the load balancer while processing requests. The default rule is not counted. The 10 free rule evaluations per request are included in this count. **Reporting criteria**: There is a nonzero value **Statistics**: The most useful statistic is `Sum`. **Dimensions** • `LoadBalancer`

The `AWS/ApplicationELB` namespace includes the following metrics for load balancer capacity units (LCU).

Metric	Description
`ConsumedLCUs`	The number of load balancer capacity units (LCU) used by your load balancer. You pay for the number of LCUs that you use per hour. When LCU reservation is active, ConsumedLCUs will report `0` if usage is below the reserved capacity, and will report values above `0` if usage exceeds the reserved LCUs. For more information, see Elastic Load Balancing pricing. **Reporting criteria**: Always reported **Statistics**: All **Dimensions** • `LoadBalancer`
`PeakLCUs`	The maximum number of load balancer capacity units (LCU) used by your load balancer at a given point in time. Only applicable when using LCU Reservation. **Reporting criteria**: Always **Statistics**: The most useful statistics are `Sum` and `Max`. **Dimensions** • `LoadBalancer`
`ReservedLCUs`	A billing metric that reports the reserved capacity on a per-minute basis. The total ReservedLCUs over any period is the amount of LCUs you will be charged for. For example, if 500 LCUs are reserved for an hour, the per-minute metric will be 8.33 LCUs. For more information, see Monitor reservation. **Reporting criteria**: There is a nonzero value **Statistics**: All **Dimensions** • `LoadBalancer`

The `AWS/ApplicationELB` namespace includes the following metrics for targets.

Metric	Description
`AnomalousHostCount`	The number of hosts detected with anomalies. **Reporting criteria**: Always reported **Statistics**: The only meaningful statistics are `Minimum` and `Maximum`. **Dimensions** • `TargetGroup`, `LoadBalancer` • `TargetGroup`, `AvailabilityZone`, `LoadBalancer`
`HealthyHostCount`	The number of targets that are considered healthy. **Reporting criteria**: Reported if there are registered targets. **Statistics**: The most useful statistics are `Average`, `Minimum`, and `Maximum`. **Dimensions** • `LoadBalancer`, `TargetGroup` • `LoadBalancer`, `AvailabilityZone`, `TargetGroup`
`HTTPCode_Target_2XX_Count`, `HTTPCode_Target_3XX_Count`, `HTTPCode_Target_4XX_Count`, `HTTPCode_Target_5XX_Count`	The number of HTTP response codes generated by the targets. This does not include any response codes generated by the load balancer. **Reporting criteria**: Reported if there are registered targets. **Statistics**: The most useful statistic is `Sum`. `Minimum`, `Maximum`, and `Average` all return 1. **Dimensions** • `LoadBalancer` • `AvailabilityZone`, `LoadBalancer` • `TargetGroup`, `LoadBalancer` • `TargetGroup`, `AvailabilityZone`, `LoadBalancer`
`MitigatedHostCount`	The number of targets under mitigation. **Reporting criteria**: Always reported **Statistics**: The most useful statistics are `Average`, `Minimum`, and `Maximum`. **Dimensions** • `TargetGroup`, `LoadBalancer` • `TargetGroup`, `AvailabilityZone`, `LoadBalancer`
`RequestCountPerTarget`	The average request count per target, in a target group. You must specify the target group using the `TargetGroup` dimension. This metric does not apply if the target is a Lambda function. This count uses the total number of requests received by the target group, divided by the number of healthy targets in the target group. If there are no healthy targets in the target group, it is divided by the total number of registered targets. **Reporting criteria**: Always reported **Statistics**: The only valid statistic is `Sum`. This represents the average not the sum. **Dimensions** • `TargetGroup` • `TargetGroup`, `AvailabilityZone` • `LoadBalancer`, `TargetGroup` • `LoadBalancer`, `AvailabilityZone`, `TargetGroup`
`TargetConnectionErrorCount`	The number of connections that were not successfully established between the load balancer and target. This metric does not apply if the target is a Lambda function. This metric is not incremented for unsuccessful health check connections. **Reporting criteria**: There is a nonzero value **Statistics**: The most useful statistic is `Sum`. **Dimensions** • `LoadBalancer` • `AvailabilityZone`, `LoadBalancer` • `TargetGroup`, `LoadBalancer` • `TargetGroup`, `AvailabilityZone`, `LoadBalancer`
`TargetResponseTime`	The time elapsed, in seconds, after the request leaves the load balancer until the target starts to send the response headers. This is equivalent to the `target_processing_time` field in the access logs. **Reporting criteria**: There is a nonzero value **Statistics**: The most useful statistics are `Average` and `pNN.NN` (percentiles). **Dimensions** • `LoadBalancer` • `AvailabilityZone`, `LoadBalancer` • `TargetGroup`, `LoadBalancer` • `TargetGroup`, `AvailabilityZone`, `LoadBalancer`
`TargetTLSNegotiationErrorCount`	The number of TLS connections initiated by the load balancer that did not establish a session with the target. Possible causes include a mismatch of ciphers or protocols. This metric does not apply if the target is a Lambda function. **Reporting criteria**: There is a nonzero value **Statistics**: The most useful statistic is `Sum`. **Dimensions** • `LoadBalancer` • `AvailabilityZone`, `LoadBalancer` • `TargetGroup`, `LoadBalancer` • `TargetGroup`, `AvailabilityZone`, `LoadBalancer`
`UnHealthyHostCount`	The number of targets that are considered unhealthy. When you deregister a target, this decreases `HealthyHostCount` but does not increase `UnhealthyHostCount`. **Reporting criteria**: Reported if there are registered targets. **Statistics**: The most useful statistics are `Average`, `Minimum`, and `Maximum`. **Dimensions** • `LoadBalancer`, `TargetGroup` • `LoadBalancer`, `AvailabilityZone`, `TargetGroup`
`ActiveZonalShiftHostCount`	The number of targets that are considered disabled due to zonal shift. **Reporting criteria**: Reported when there is a value **Statistics**: The most useful statistic is `Sum`. **Dimensions** • `LoadBalancer`, `TargetGroup`. • `AvailabilityZone`, `LoadBalancer`, `TargetGroup`.

The `AWS/ApplicationELB` namespace includes the following metrics for target group health. For more information, see Target group health.

Metric	Description
`HealthyStateDNS`	The number of zones that meet the DNS healthy state requirements. **Statistics**: The most useful statistic is `Max`. **Dimensions** • `LoadBalancer`, `TargetGroup` • `AvailabilityZone`, `LoadBalancer`, `TargetGroup`
`HealthyStateRouting`	The number of zones that meet the routing healthy state requirements. **Statistics**: The most useful statistic is `Max`. **Dimensions** • `LoadBalancer`, `TargetGroup` • `AvailabilityZone`, `LoadBalancer`, `TargetGroup`
`UnhealthyRoutingRequestCount`	The number of requests that are routed using the routing failover action (fail open). **Statistics**: The most useful statistic is `Sum`. **Dimensions** • `LoadBalancer`, `TargetGroup` • `AvailabilityZone`, `LoadBalancer`, `TargetGroup`
`UnhealthyStateDNS`	The number of zones that do not meet the DNS healthy state requirements and therefore were marked unhealthy in DNS. **Statistics**: The most useful statistic is `Min`. **Dimensions** • `LoadBalancer`, `TargetGroup` • `AvailabilityZone`, `LoadBalancer`, `TargetGroup`
`UnhealthyStateRouting`	The number of zones that do not meet the routing healthy state requirements, and therefore the

UnhealthyStateRouting	The number of zones that do not meet the routing healthy state requirements, and therefore the load balancer distributes traffic to all targets in the zone, including the unhealthy targets. **Statistics**: The most useful statistic is `Min`. **Dimensions** - `LoadBalancer`, `TargetGroup` - `AvailabilityZone`, `LoadBalancer`, `TargetGroup`

The `AWS/ApplicationELB` namespace includes the following metrics for Lambda functions that are registered as targets.

Metric	Description
`LambdaInternalError`	The number of requests to a Lambda function that failed because of an issue internal to the load balancer or AWS Lambda. To get the error reason codes, check the error_reason field of the access log. **Reporting criteria**: There is a nonzero value **Statistics**: The only meaningful statistic is `Sum`. **Dimensions** - `TargetGroup` - `TargetGroup`, `LoadBalancer`
`LambdaTargetProcessedBytes`	The total number of bytes processed by the load balancer for requests to and responses from a Lambda function. **Reporting criteria**: There is a nonzero value **Statistics**: The only meaningful statistic is `Sum`. **Dimensions** - `LoadBalancer`
`LambdaUserError`	The number of requests to a Lambda function that failed because of an issue with the Lambda function. For example, the load balancer did not have permission to invoke the function, the load balancer received JSON from the function that is malformed or missing required fields, or the size of the request body or response exceeded the maximum size of 1 MB. To get the error reason codes, check the error_reason field of the access log. **Reporting criteria**: There is a nonzero value **Statistics**: The only meaningful statistic is `Sum`. **Dimensions** - `TargetGroup` - `TargetGroup`, `LoadBalancer`

The `AWS/ApplicationELB` namespace includes the following metrics for user authentication.

Metric	Description
`ELBAuthError`	The number of user authentications that could not be completed because an authenticate action was misconfigured, the load balancer couldn't establish a connection with the IdP, or the load balancer couldn't complete the authentication flow due to an internal error. To get the error reason codes, check the error_reason field of the access log. **Reporting criteria**: There is a nonzero value **Statistics**: The only meaningful statistic is `Sum`. **Dimensions** - `LoadBalancer` - `AvailabilityZone`, `LoadBalancer`
`ELBAuthFailure`	The number of user authentications that could not be completed because the IdP denied access to the user or an authorization code was used more than once. To get the error reason codes, check the error_reason field of the access log. **Reporting criteria**: There is a nonzero value **Statistics**: The only meaningful statistic is `Sum`. **Dimensions** - `LoadBalancer` - `AvailabilityZone`, `LoadBalancer`
`ELBAuthLatency`	The time elapsed, in milliseconds, to query the IdP for the ID token and user info. If one or more of these operations fail, this is the time to failure. **Reporting criteria**: There is a nonzero value **Statistics**: All statistics are meaningful. **Dimensions** - `LoadBalancer` - `AvailabilityZone`, `LoadBalancer`
`ELBAuthRefreshTokenSuccess`	The number of times the load balancer successfully refreshed user claims using a refresh token provided by the IdP. **Reporting criteria**: There is a nonzero value **Statistics**: The only meaningful statistic is `Sum`. **Dimensions** - `LoadBalancer` - `AvailabilityZone`, `LoadBalancer`
`ELBAuthSuccess`	The number of authenticate actions that were successful. This metric is incremented at the end of the authentication workflow, after the load balancer has retrieved the user claims from the IdP. **Reporting criteria**: There is a nonzero value **Statistics**: The most useful statistic is `Sum`. **Dimensions** - `LoadBalancer` - `AvailabilityZone`, `LoadBalancer`
`ELBAuthUserClaimsSizeExceeded`	The number of times that a configured IdP returned user claims that exceeded 11K bytes in size. **Reporting criteria**: There is a nonzero value **Statistics**: The only meaningful statistic is `Sum`. **Dimensions** - `LoadBalancer` - `AvailabilityZone`, `LoadBalancer`

The `AWS/ApplicationELB` namespace includes the following metrics for target optimizer.

Metric	Description
`TargetControlRequestCount`	Number of requests forwarded by ALB to agents. **Reporting criteria**: Target optimizer is enabled on a target group and there is a nonzero value. **Statistics**: The only meaningful statistic is `Sum`. **Dimensions** - `LoadBalancer` - `AvailabilityZone`, `LoadBalancer`
`TargetControlRequestRejectCount`	Number of requests rejected by ALB due to no targets being ready to receive requests. This metric shows an uptick when TargetControlWorkQueueLength is zero. **Reporting criteria**: Target optimizer is enabled on a target group and there is a nonzero value. **Statistics**: The only meaningful statistic is `Sum`. **Dimensions** - `LoadBalancer` - `AvailabilityZone`, `LoadBalancer`
`TargetControlActiveChannelCount`	Number of active control channels between ALB and agents. For a load balancer, this should be equal to the number of agents. A lower than expected number indicates that agents are not configured properly or are not available. **Reporting criteria**: Target optimizer is enabled on a target group and there is a nonzero value. **Statistics**: The only meaningful statistic is `Sum`. **Dimensions** - `LoadBalancer` - `AvailabilityZone`, `LoadBalancer`
`TargetControlNewChannelCount`	Number of new control channels created between ALB and agents. You will see an uptick in this metric when a new target with the agent installed is successfully added to the target group. **Reporting criteria**: Target optimizer is enabled on a target group and there is a nonzero value. **Statistics**: The only meaningful statistic is `Sum`. **Dimensions** - `LoadBalancer` - `AvailabilityZone`, `LoadBalancer`
`TargetControlChannelErrorCount`	Number of control channels between ALB and agents that failed to establish or experienced an unexpected error. A control channel error will result in that agent (and target) not receiving any application traffic. **Reporting criteria**: Target optimizer is enabled on a target group and there is a nonzero value. **Statistics**: The only meaningful statistic is `Sum`. **Dimensions** - `LoadBalancer` - `AvailabilityZone`, `LoadBalancer`
`TargetControlWorkQueueLength`	Number of signals received by the ALB from agents asking for requests. This data comes from snapshots taken at 1-minute intervals. Sub-minute changes are not captured. **Reporting criteria**: Target optimizer is enabled on a target group and there is a nonzero value. **Statistics**: The only meaningful statistic is `Sum`. **Dimensions** - `LoadBalancer` - `AvailabilityZone`, `LoadBalancer`
`TargetControlProcessedBytes`	Number of bytes processed by ALB for traffic to target groups that enable target optimizer. **Reporting criteria**: Target optimizer is enabled on a target group and there is a nonzero value. **Statistics**: The most meaningful statistic is `Sum`. **Dimensions** - `LoadBalancer` - `AvailabilityZone`, `LoadBalancer`

Metric dimensions for Application Load Balancers

To filter the metrics for your Application Load Balancer, use the following dimensions.

Dimension	Description
`AvailabilityZone`	Filters the metric data by Availability Zone.
`LoadBalancer`	Filters the metric data by load balancer. Specify the load balancer as follows: app/*load-balancer-name*/*1234567890123456* (the final portion of the load balancer ARN).
`TargetGroup`	Filters the metric data by target group. Specify the target group as follows: targetgroup/*target-group-name*/*1234567890123456* (the final portion of the target group ARN).

Statistics for Application Load Balancer metrics

CloudWatch provides statistics based on the metric data points published by Elastic Load Balancing. Statistics are metric data aggregations over specified period of time. When you request statistics, the returned data stream is identified by the metric name and dimension. A dimension is a name-value pair that uniquely identifies a metric. For example, you can request statistics for all the healthy EC2 instances behind a load balancer launched in a specific Availability Zone.

The `Minimum` and `Maximum` statistics reflect the minimum and maximum values of the data points reported by the individual load balancer nodes in each sampling window. For example, suppose there are 2 load balancer nodes that make up the Application Load Balancer. One node has `HealthyHostCount` with a `Minimum` of 2, a `Maximum` of 10, and an `Average` of 6, while the other node has `HealthyHostCount` with a `Minimum` of 1, a `Maximum` of 5, and an `Average` of 3. Therefore, the load balancer has a `Minimum` of 1, a `Maximum` of 10, and an `Average` of about 4.

We recommend you monitor for non-zero `UnHealthyHostCount` in the `Minimum` statistic, and alarm on non-zero value for more than one data point. Using the `Minimum` will detect when targets are considered unhealthy by every node and Availability Zone of your load balancer. Alarming on `Average` or `Maximum` is useful if you want to be alerted to potential problems, and we recommend customers review this metric and investigate non-zero occurrences. Mitigating failures automatically can be done following best practices of using load balancer health check in Amazon EC2 Auto Scaling, or Amazon Elastic Container Service (Amazon ECS).

The `Sum` statistic is the aggregate value across all load balancer nodes. Because metrics include multiple reports per period, `Sum` is only applicable to metrics that are aggregated across all load balancer nodes.

The `SampleCount` statistic is the number of samples measured. Because metrics are gathered based on sampling intervals and events, this statistic is typically not useful. For example, with `HealthyHostCount`, `SampleCount` is based on the number of samples that each load balancer node reports, not the number of healthy hosts.

A percentile indicates the relative standing of a value in a data set. You can specify any percentile, using up to two decimal places (for example, p95.45). For example, the 95th percentile means that 95 percent of the data is below this value and 5 percent is above. Percentiles are often used to isolate anomalies. For example, suppose that an application serves the majority of requests from a cache in 1-2 ms, but in 100-200 ms if the cache is empty. The maximum reflects the slowest case, around 200 ms. The average doesn't indicate the distribution of the data. Percentiles provide a more meaningful view of the application's performance. By using the 99th percentile as an Auto Scaling trigger or a CloudWatch alarm, you can target that no more than 1 percent of requests take longer than 2 ms to process.

View CloudWatch metrics for your load balancer

You can view the CloudWatch metrics for your load balancers using the Amazon EC2 console. These metrics are displayed as monitoring graphs. The monitoring graphs show data points if the load balancer is active and receiving requests.

Alternatively, you can view metrics for your load balancer using the CloudWatch console.

To view metrics using the console

1. Open the Amazon EC2 console at https://console.aws.amazon.com/ec2/.
2. To view metrics filtered by target group, do the following:
   a. In the navigation pane, choose **Target Groups**.
   b. Select your target group, and then choose the **Monitoring** tab.
   c. (Optional) To filter the results by time, select a time range from **Showing data for**.
   d. To get a larger view of a single metric, select its graph.
3. To view metrics filtered by load balancer, do the following:
   a. In the navigation pane, choose **Load Balancers**.
   b. Select your load balancer, and then choose the **Monitoring** tab.
   c. (Optional) To filter the results by time, select a time range from **Showing data for**.
   d. To get a larger view of a single metric, select its graph.

To view metrics using the CloudWatch console

1. Open the CloudWatch console at https://console.aws.amazon.com/cloudwatch/.
2. In the navigation pane, choose **Metrics**.
3. Select the **ApplicationELB** namespace.
4. (Optional) To view a metric across all dimensions, enter its name in the search field.
5. (Optional) To filter by dimension, select one of the following:
   - To display only the metrics reported for your load balancers, choose **Per AppELB Metrics**. To view the metrics for a single load balancer, enter its name in the search field.
   - To display only the metrics reported for your target groups, choose **Per AppELB, per TG Metrics**. To view the metrics for a single target group, enter its name in the search field.
   - To display only the metrics reported for your load balancers by Availability Zone, choose **Per AppELB, per AZ Metrics**. To view the metrics for a single load balancer, enter its name in the search field. To view the metrics for a single Availability Zone, enter its name in the search field.
   - To display only the metrics reported for your load balancers by Availability Zone and target group, choose **Per AppELB, per AZ, per TG Metrics**. To view the metrics for a single load balancer, enter its name in the search field. To view the metrics for a single target group, enter its name in the search field. To view the metrics for a single Availability Zone, enter its name in the search field.

To view metrics using the AWS CLI

Use the following list-metrics command to list the available metrics:

```
aws cloudwatch list-metrics --namespace AWS/ApplicationELB
```

To get the statistics for a metric using the AWS CLI

Use the following get-metric-statistics command get statistics for the specified metric and dimension. CloudWatch treats each unique combination of dimensions as a separate metric. You can't retrieve statistics using combinations of dimensions that were not specially published. You must specify the same dimensions that were used when the metrics were created.

```
aws cloudwatch get-metric-statistics --namespace AWS/ApplicationELB \
--metric-name UnHealthyHostCount --statistics Average  --period 3600 \
--dimensions Name=LoadBalancer,Value=app/my-load-balancer/50dc6c495c0c9188 \
Name=TargetGroup,Value=targetgroup/my-targets/73e2d6bc24d8a067 \
--start-time 2016-04-18T00:00:00Z --end-time 2016-04-21T00:00:00Z
```

The following is example output:

```
{
    "Datapoints": [
        {
            "Timestamp": "2016-04-18T22:00:00Z",
            "Average": 0.0,
            "Unit": "Count"
        },
        {
            "Timestamp": "2016-04-18T04:00:00Z",
            "Average": 0.0,
            "Unit": "Count"
        },
        ...
    ],
    "Label": "UnHealthyHostCount"
}
```

View related pages ✦ Abstracts generated by AI

< 1 2 >

Elasticloadbalancing › network
CloudWatch metrics for your Network Load Balancer...
Monitor NLB CloudWatch metrics tracking TLS failures, healthy targets, LCU costs, security group rejected flows via EC2 console.
July 1, 2026

Lightsail › userguide
Configure health check settings for Lightsail load balancers...
Configure Lightsail load balancer health checks, customize paths, diagnose metrics, track host counts, HTTP errors, response times.
June 30, 2026

Elasticloadbalancing › classic
CloudWatch metrics for your Classic Load Balancer...
Monitor Classic Load Balancer CloudWatch metrics: healthy/unhealthy hosts, surge queue, spillover, HTTP response codes, Availability Zone filters.
July 1, 2026

Discover highly rated pages ✦ Abstracts generated by AI

< 1 2 3 4 >

Elasticloadbalancing › application
What is an Application Load Balancer?
Distributes traffic across targets; listener rules, health checks, path routing, migration benefits, related services integration.
July 1, 2026

Elasticloadbalancing › application
Create an HTTPS listener for your Application Load Balancer...
Configure HTTPS listener ALB deploying SSL certificate, security policy, mTLS, forwarding traffic target groups.
July 1, 2026

Elasticloadbalancing › application
Target groups for your Application Load Balancers...
Configure Application Load Balancer target groups, health checks, DNS failover, gRPC rules, cross-zone thresholds.
July 1, 2026

Next topic: Access logs
Previous topic: Monitor your load balancers

Privacy | Site terms | Cookie preferences | © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.